

Documentación de Estructuras de Datos Utilizadas

1. Introducción y Justificación General

Para el desarrollo del sistema de Monopoly Distribuido, se implementaron estructuras de datos lineales propias desde cero en el lenguaje C#, respetando la restricción de no utilizar las colecciones genéricas del sistema (como `List<T>`, `LinkedList<T>`, `Queue<T>`, etc.).

La arquitectura interna se basa en el concepto de Nodos Genéricos (`NodoSimple<T>` y `NodoDoble<T>`), lo cual garantiza una alta reutilización de código, desacoplamiento de la lógica del juego y tipado fuerte (type safety).

2. Descripción Detallada de las Estructuras

2.1 Tablero de Juego: Lista Circular Doblemente Enlazada (`ListaCircularDoble<T>`)

- **Definición y Estructura:** Es una estructura de datos lineal compuesta por nodos doblemente enlazados (`NodoDoble<T>`), donde cada nodo posee un puntero hacia el nodo Siguiente y otro hacia el Anterior. Su característica distintiva es que el último nodo se enlaza directamente con el primer nodo (Cabeza), formando una cadena cerrada o bucle.
- **Nodo Asociado:** `NodoDoble<Casilla>` (contiene la casilla del tablero).
- **Justificación de Uso en el Proyecto:** El tablero de Monopoly consta de 24 casillas. Una lista circular doble es ideal por tres razones principales:
 1. **Naturaleza Cíclica del Tablero:** Permite que los jugadores den múltiples vueltas continuas al tablero de forma fluida. Al pasar por la última casilla, el puntero Siguiente redirige de inmediato a la casilla de Inicio sin requerir reinicios manuales de índices.
 2. **Desplazamiento Bidireccional:** Permite que las cartas de eventos u otras dinámicas muevan a los jugadores tanto hacia adelante (usando Siguiente) como hacia atrás (usando Anterior).

- **Operaciones Principales e Implementación:**

- Agregar(T dato): Inserta una casilla al tablero manteniendo los enlaces circulares y bidireccionales en $O(1)$.
- Mover(NodoDoble<T> nodoInicio, int pasos): Recibe la posición actual del jugador y la suma de los dados. Realiza iteraciones hacia adelante ($pasos > 0$) o hacia atrás ($pasos < 0$) retornando el nodo destino exacto.

2.2 Administración de Turnos: Cola Circular (ColaCircular<T>)

- **Definición y Estructura:** Es una variante de la cola implementada internamente sobre nodos simples (NodoSimple<T>). Mantiene una referencia al puntero ultimo, cuyo puntero ultimo.Siguiente apunta siempre al frente de la cola (el primer jugador en la secuencia).
- **Nodo Asociado:** NodoSimple<Jugador>.
- **Justificación de Uso en el Proyecto:** El sistema de turnos exige una administración estrictamente secuencial y continua entre los 4 jugadores activos:
 1. **Garantía de Turno:** Impide que los jugadores actúen fuera de su turno asignado, respondiendo a las validaciones del servidor.
 2. **Rotación Eficiente:** Cuando un jugador finaliza su turno, la estructura no elimina al jugador, sino que simplemente desplaza el puntero del turno activo hacia el siguiente participante en $O(1)$.
- **Operaciones Principales e Implementación:**
 - Encolar(T dato): Registra a los jugadores al iniciar la partida en $O(1)$.
 - AvanzarTurno(): Rota la referencia del puntero al siguiente nodo (ultimo = ultimo.Siguiente), permitiendo reiniciar el ciclo de turnos de manera limpia y sin reconstruir la estructura.
 - VerSiguiente(): Retorna el jugador que posee el turno actual sin modificar el estado de la cola.

2.3 Historial de Transacciones: Lista Doblemente Enlazada (ListaDoble<T>)

- **Definición y Estructura:** Es una lista lineal constituida por nodos doblemente enlazados (NodoDoble<T>) que mantiene dos referencias explícitas: el nodo inicial (Cabeza) y el nodo final (Cola). A diferencia de la lista circular, sus extremos no se conectan entre sí (los punteros inicial y final son null).
- **Nodo Asociado:** NodoDoble<Transaccion>.
- **Justificación de Uso en el Proyecto:** El pliego de requerimientos exige almacenar de forma cronológica todas las operaciones económicas (compras, alquileres, eventos, premios) y permitir lecturas completas en ambos sentidos:
 1. **Lectura desde la más antigua:** Se realiza un recorrido natural desde Cabeza hasta Cola.
 2. **Lectura desde la más reciente:** Gracias al método ObtenerEnOrdenInverso() y los enlaces Anterior, se puede recorrer la lista comenzando en Cola en dirección a Cabeza, facilitando la consulta de los últimos movimientos realizados en la partida.
 3. **Exportación a Reportes:** Permite iterar fácilmente toda la bitácora económica para generar el archivo de reporte .TXT obligatorio.

Operaciones Principales e Implementación:

- **Agregar(T dato):** Añade un nuevo registro de transacción al final del historial en $O(1)$.
- **ObtenerEnOrdenInverso():** Utiliza la instrucción yield return comenzando en la Cola para iterar eficientemente hacia atrás sin consumir memoria adicional.
- **GetEnumerator():** Implementa la interfaz IEnumerable<T> para permitir la búsqueda por jugador o por tipo mediante recorridos lineales.

2.4 Propiedades del Jugador: Lista Simplemente Enlazada (ListaSimple<T>)

- **Definición y Estructura:** Es una estructura lineal unidireccional formada por nodos simples (NodoSimple<T>). Cada nodo almacena la referencia al objeto del dato y un puntero único hacia el nodo Siguiente.
- **Nodo Asociado:** NodoSimple<Propiedad>.
- **Justificación de Uso en el Proyecto:** Cada objeto de tipo Jugador requiere mantener su propio inventario individual de inmuebles adquiridos:
 1. **Bajo Consumo de Memoria:** Al requerir únicamente un puntero unidireccional por nodo, es ideal para colecciones pequeñas y de tamaño dinámico.
 2. **Gestión de Modificaciones:** Permite iterar sobre el patrimonio del jugador para calcular su valor neto al final de la partida, así como eliminar o transferir propiedades de manera sencilla si un jugador cae en bancarrota.
- **Operaciones Principales e Implementación:**
 - Agregar(T dato): Conecta una nueva propiedad adquirida al final de la lista del jugador.
 - Eliminar(T dato): Recorre los nodos buscando la propiedad a retirar, reajustando el puntero Siguiente del nodo previo para desvincularla en caso de pérdida o bancarrota.